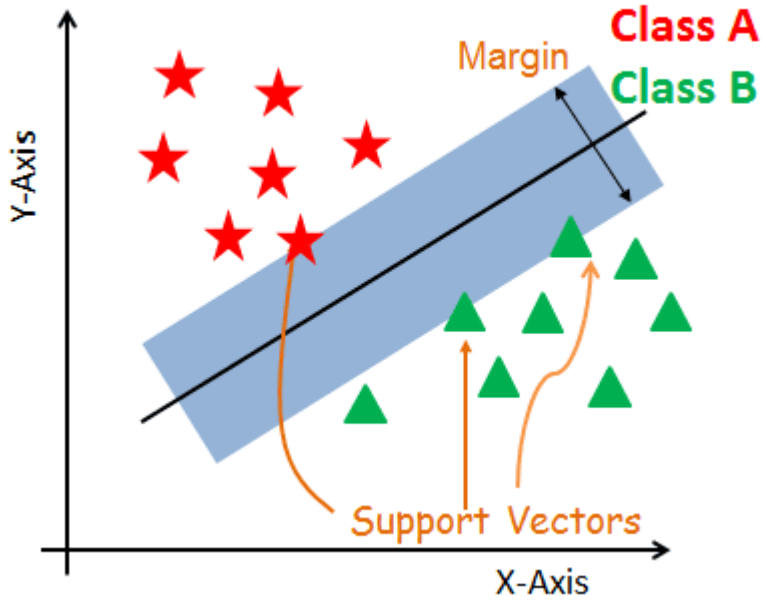
Support Vector Machine (SVM)

A **Support Vector Machine** or **SVM** is a machine learning algorithm that looks at data and sorts it into one of two categories.

Support Vector Machine is a supervised and linear Machine Learning algorithm most commonly used for solving classification problems and is also referred to as Support Vector Classification.



1 Why does maximizing margin improve generalization? Relate to VC dimension.

The Vapnik-Chervonenkis (VC) dimension measures the complexity or capacity of a hypothesis space. A higher VC dimension allows a model to fit complex patterns but increases the risk of overfitting.

Maximizing the geometric margin reduces the capacity of the hypothesis space. For linear classifiers, the VC dimension is bounded by:

$$VC \leq \min(R^2 / \gamma^2, d) + 1$$

where R is data radius and γ is margin. A larger margin (γ ↑) reduces the effective VC dimension, lowering overfitting risk and improving generalization.

2 Derive the dual form of SVM using Lagrange multipliers.

To derive the dual problem of the support vector machine (SVM) from the primal problem using the Lagrange multiplier method, we start with the primal problem formulation of SVM:

Primal Problem: Minimize

$$1/2 * ||w||^2 + C * \Sigma\xi_i$$

Subject to:

$$y_i * (w^T * x_i + b) >= 1 - \xi_i$$
$$\xi_i >= 0$$

where:

- w is the weight vector
- b is the bias term

- C is the regularization parameter
- x_i is the input vector
- y_i is the corresponding label
- ξ_i is the slack variable

To derive the dual problem, we introduce Lagrange multipliers α_i for each constraint and form the Lagrangian:

Lagrangian:

$$L(w, b, \xi, \alpha) = 1/2 * ||w||^2 + C * \Sigma\xi_i - \Sigma\alpha_i * (y_i * (w^T * x_i + b) - 1 + \xi_i) - \Sigma\mu_i * \xi_i$$

where:

- α_i is the Lagrange multiplier for the inequality constraint
- μ_i is the Lagrange multiplier for the non-negativity constraint

To find the dual problem, we need to maximize the Lagrangian with respect to w, b, and ξ, and minimize it with respect to α and μ.

Taking the derivatives with respect to w, b, and ξ, and setting them to zero, we get:

$$\partial L/\partial w = w - \Sigma\alpha_i * y_i * x_i = 0 \Rightarrow w = \Sigma\alpha_i * y_i * x_i$$
$$\partial L/\partial b = -\Sigma\alpha_i * y_i = 0 \Rightarrow \Sigma\alpha_i * y_i = 0$$
$$\partial L/\partial \xi_i = C - \alpha_i - \mu_i = 0 \Rightarrow \alpha_i + \mu_i = C$$

Substituting these back into the Lagrangian, we obtain the dual problem:

Dual Problem: Maximize:

$$W(\alpha) = \Sigma\alpha_i - 1/2 * \Sigma\Sigma\alpha_i * \alpha_j * y_i * y_j * x_i^T * x_j$$

Subject to:

$\Sigma \alpha_i * y_i = 0$
$0 <= \alpha_i <= C$

where:

- $W(\alpha)$ is the dual objective function
- The solution to the dual problem gives us the optimal values of the Lagrange multipliers α_i. From these values, we can compute the weight vector w and the bias term b using the equations derived earlier. The support vectors are the data points corresponding to non-zero α_i values.
- The dual problem allows us to solve the SVM optimization problem using the kernel trick, where the inner products $x_i^T * x_j$ are replaced with a kernel function $K(x_i, x_j)$. This enables SVM to work in high-dimensional feature spaces without explicitly computing the transformed feature vectors.
- Note that the dual problem is typically solved using optimization algorithms such as Sequential Minimal Optimization (SMO) or Quadratic Programming (QP).

3 Why do only support vectors matter in the final solution?

These support vectors are important because they define the decision boundary and have the most influence on the classification process.

Only support vectors matter because, from the KKT conditions, training points lying outside the margin have

$$\alpha_i = 0$$

and therefore do not appear in the dual representation

$$w = \sum \alpha_i y_i x_i$$

Hence the SVM decision boundary depends exclusively on points on or inside the margin (the support vectors).

4 Compare hinge loss vs logistic loss mathematically.

Hinge loss and logistic loss are both convex loss functions used for binary classification, but they differ in their optimization goals, margin requirements, and mathematical properties.

Hinge Loss

The use of hinge loss is very common in binary classification problems where we want to separate a group of data points from those from another group. It is a convex, piecewise-linear loss used in Support Vector Machines that enforces a margin of 1. The loss becomes zero once the margin exceeds 1.

$$\ell_{\text{hinge}}(m) = \max(0, 1 - m), \quad m = yf(x)$$

Logistic Loss

Logistic loss (also known as the cross-entropy loss and log loss) , it is a smooth, convex loss derived from the negative log-likelihood of logistic regression. The loss decreases smoothly as the margin increases and never becomes exactly zero.

$$\ell_{\text{logistic}}(m) = \log(1 + e^{-m})$$

5 Prove convexity of the SVM objective.

A function is said to be convex if the line segment between any two points on its graph lies above or on the graph itself. In simple terms, a convex function curves upward and does not have multiple valleys or dips.

A function $f(x)$ is convex if :

$$f(\lambda x_1 + (1-\lambda)x_2) \leq \lambda f(x_1) + (1-\lambda) f(x_2)$$

for all $x1$,$x2$ and $\lambda \in [\,0\,,\,1\,]$.

SVM Objective Function

Soft-margin SVM:

$$\min_{w,b} \quad J(w,b) = \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \max\left(0,\ 1 - y_i(w^T x_i + b)\right)$$

This minimized function $J(w,b)$ is called the **objective function**.

Step 1: Regularization Term

$$f_1(w) = \frac{1}{2}\|w\|^2 = \frac{1}{2}w^T w$$

Hessian:

$$\nabla^2 f_1(w) = I \succeq 0$$

$$\Rightarrow f_1(w) \text{ is convex}$$

Step 2: Hinge Loss Term

For each sample:

$$f_i(w,b) = \max\left(0,\ 1 - y_i(w^T x_i + b)\right)$$

Let

$$g_i(w,b) = 1 - y_i(w^T x_i + b)$$

Since affine functions are convex:

$$g_i(w,b) \text{ is convex}$$

Property of max:

$$\max(0,\ g_i(w,b)) \text{ is convex}$$
$$\Rightarrow f_i(w,b) \text{ is convex}$$

Step 3: Sum of Hinge Losses

$$f_2(w,b) = C\sum_{i=1}^{n} f_i(w,b)$$

Since $C \geq 0$:

$$f_2(w,b) \text{ is convex}$$

Step 4: Total Objective

$$J(w,b) = f_1(w) + f_2(w,b)$$

Sum of convex functions is convex:

$$\boxed{J(w,b) \text{ is convex}}$$

6 How does kernel choice affect feature space geometry?

Kernel choice changes the geometry of the feature space by altering the implicit mapping $\phi(x)$, which determines the shape of the SVM decision boundary and margin.

7 What happens when C → ∞ ?

SVM Objective

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i$$

Subject to:

$$y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

As $C \to \infty$

$$C\sum \xi_i \gg \frac{1}{2}\|w\|^2$$

The optimizer strongly penalizes any slack.

As C→∞, SVM prioritizes zero training error, approaching hard-margin SVM and increasing overfitting risk.

8 Derive SVM from structural risk minimization theory.

Structural Risk Minimization is a principle that minimizes an upper bound on the expected risk by jointly controlling empirical error and model capacity.

SRM risk bound:

$$R(f) \leq R_{emp}(f) + \Omega(h, n, \delta)$$

$R(f)$: True (expected) risk — probability of misclassification on unseen data.
$R_{emp}(f)$: Empirical risk — training error.
$\Omega(h, n, \delta)$: Confidence/complexity term controlling overfitting.
h: VC dimension — capacity of hypothesis space.
n: Number of training samples.
δ: Confidence level.

SRM objective:

$$\min \left(\text{empirical risk} + \text{model complexity}\right)$$

For linear classifiers with margin γ:

$$R(f) \leq R_{emp}(f) + O\left(\frac{R^2}{\gamma^2 n}\right)$$

γ: Geometric margin — distance between separating hyperplane and nearest points.
R: Radius of the smallest sphere enclosing the data.

Thus SRM implies maximizing margin while minimizing training error.

Linear classifier:

$$f(x) = \text{sign}(w^T x + b)$$

w: Weight vector — orientation of hyperplane.
b: Bias — offset of hyperplane.

Geometric margin:

$$\gamma = \frac{2}{\|w\|}$$

Maximizing margin is equivalent to minimizing $\|w\|$:

$$\min_{w,b} \ \frac{1}{2}\|w\|^2$$

To enforce correct classification:

$$y_i(w^T x_i + b) \geq 1, \quad \forall i$$

x_i: Feature vector of sample i.
y_i: Class label (± 1).
i: Sample index.

Hard-margin SVM:

$$\min_{w,b} \ \frac{1}{2}\|w\|^2 \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1$$

For non-separable data, introduce slack variables:

$$y_i(w^T x_i + b) \geq 1 - \xi_i, \qquad \xi_i \geq 0$$

ξ_i: Slack variable — amount of margin violation.

Empirical risk surrogate:

$$R_{emp} \approx \sum_{i=1}^{n} \xi_i$$

SRM trade-off formulation:

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i$$

Subject to:

$$y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

C: Regularization parameter — controls trade-off between margin maximization and training error.

Since

$$\xi_i = \max(0,\ 1 - y_i(w^T x_i + b))$$

the objective becomes

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \max(0,\ 1 - y_i(w^T x_i + b))$$

Hinge loss: convex loss measuring margin violation.

Conclusion: SVM is obtained from Structural Risk Minimization by maximizing the margin (capacity control) while penalizing empirical errors using slack variables.

9 Compare SVM to neural networks in high-dimensional regimes.

Aspect	Support Vector Machines (SVM)	Neural Networks (NN)
Basic idea	SVM is a powerful machine learning algorithm used for linear and nonlinear classification, regression, and outlier detection by finding an optimal separating hyperplane.	Neural networks are brain-inspired models made of interconnected neurons arranged in layers that learn by adjusting weights and biases to minimize prediction error.
Learning principle	Based on margin maximization and structural risk minimization.	Based on empirical risk minimization via gradient-based learning.

Handling large datasets	Large datasets are generally **not ideal** because kernel matrix computation grows rapidly with the number of samples.	Handles large datasets effectively, especially with mini-batch training, GPUs, and distributed computing.
High-dimensional regimes (many features, few samples)	Performs very well; margin control helps generalization even when feature dimension is high.	More prone to overfitting in small-sample, high-dimension settings unless carefully regularized.
Parameter growth	Number of parameters increases roughly linearly with input dimension (through weight vector or support vectors).	Number of parameters depends mainly on network architecture; does not necessarily grow linearly with input size.
Model storage	After training, SVM retains only **support vectors** (points near the decision boundary).	Neural networks store learned information in **weights and biases** across all layers.
Feature learning	Relies heavily on good feature engineering and kernel choice.	Learns hierarchical features automatically from raw data.
Nonlinearity handling	Achieved via kernel functions (linear, polynomial, RBF, etc.) that map data into higher-dimensional spaces.	Naturally models complex nonlinear relationships through multiple nonlinear layers—no explicit kernel needed.
Optimization	Convex optimization → guarantees global optimum and stable solution.	Non-convex optimization → may converge to local minima; training is more sensitive to initialization and tuning.
Performance on unstructured data (images, audio, text embeddings)	Usually limited unless features are carefully engineered.	Excels due to deep representation learning.
Probabilistic output	Not naturally probabilistic; requires calibration (e.g., Platt scaling).	Can directly output probabilities (e.g., softmax, sigmoid).

10 When does SVM fail in practice?

- **Very large datasets (large n)**
  Training becomes slow and memory-intensive because the kernel matrix grows roughly quadratically with the number of samples.
- **Highly noisy data**
  Noise near the decision boundary creates many support vectors, leading to unstable margins and poor generalization.
- **Strong class overlap**
  When classes are not well separable, SVM struggles to find a clear margin and performance degrades.
- **Poor kernel choice or parameter tuning**
  An inappropriate kernel or badly chosen hyperparameters (C, kernel width) can cause severe underfitting or overfitting.
- **Severely imbalanced datasets**
  Standard SVM treats classes symmetrically, so the decision boundary may be biased toward the majority class, hurting minority-class detection (important in medical problems).
- **Extremely complex hierarchical patterns**
  For data like images, speech, or video, SVMs usually underperform compared to deep neural networks that can learn multi-level features.
- **When well-calibrated probabilities are required**
  SVM outputs margins, not probabilities, and needs extra calibration steps.
- **Too many irrelevant or redundant features**
  Distance-based kernels become less meaningful, which can degrade performance.